ПРАКТИЧЕСКАЯ РАБОТА №4

ОТЛАДКА И ТЕСТИРОВАНИЕ ВНЕДРЯЕМОГО ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

Тема работы

Разработка сценариев тестирования. Проведение и документирование тестирования. Доработка программного обеспечения по итогам тестирования.

Цель работы

Научиться создавать и воспроизводить контролируемые дефекты, оформлять баг-репорты, использовать Debug в Visual Studio, определять причину ошибки, исправлять код и подтверждать исправление повторным тестированием.

Задачи работы

1. Проверить исходную рабочую версию WPF-приложения из ПР №3.

2. Последовательно создать 5 контролируемых дефектов.

3. Для каждого дефекта выполнить заданный тестовый сценарий.

4. Зафиксировать ожидаемый и фактический результаты.

5. Сделать скриншоты проявления ошибок.

6. Оформить 5 баг-репортов.

7. Для одного дефекта выполнить полноценную отладку с breakpoint.

8. Исправить каждый дефект.

9. Выполнить Retest всех пяти дефектов.

10. Заполнить итоговую Debug-таблицу.

Оборудование и ПО

Visual Studio, C#, .NET, WPF. Используется проект «Сервис бронирования компьютерных аудиторий» из ПР №3.

1. Порядок выполнения

Каждый баг выполняйте отдельно. Не создавайте сразу все пять ошибок: иначе дефекты могут влиять друг на друга.

Схема: внести BUG → воспроизвести → сделать скриншот → заполнить баг-репорт → найти причину → исправить → Retest → перейти к следующему BUG.

2. Подготовка

1. Сделайте копию папки проекта ClassroomBooking и назовите её ClassroomBooking_BACKUP.

2. Откройте основной проект.

3. Выберите «Сборка» → «Пересобрать решение».

4. Запустите приложение Ctrl+F5.

5. Создайте обычное корректное бронирование и убедитесь, что приложение работает.

Скриншот №1. Исходная рабочая версия приложения.

[ВСТАВЬТЕ СКРИНШОТ №1]

3. BUG-01 — пересекающиеся бронирования

Файл: Services → BookingService.cs. Метод: AddBooking().

Замените корректное условие проверки пересечения на ошибочное:

```
bool conflict = bookings.Any(x =>
x.RoomId == booking.RoomId &&
x.Date.Date == booking.Date.Date &&
x.StartTime < booking.StartTime &&
booking.EndTime < x.EndTime);
```

1. Создайте бронирование аудитории 301 на 10:00–12:00.

2. Создайте второе бронирование той же аудитории и даты на 11:00–13:00.

3. Сравните результат с ожидаемым.

Ожидается: второе бронирование отклонено.

Фактически: _____

Скриншот №2. BUG-01 до исправления.

[ВСТАВЬТЕ СКРИНШОТ №2]

Исправьте условие:

```
bool conflict = bookings.Any(x =>
x.RoomId == booking.RoomId &&
x.Date.Date == booking.Date.Date &&
x.StartTime < booking.EndTime &&
booking.StartTime < x.EndTime);
```

BUG-01: _____

4. BUG-02 — неправильный интервал времени

Файл: MainWindow.xaml.cs. Метод: BookButton_Click().

Временно закомментируйте проверку:

```
// if (startTime >= endTime)
// {
//     StatusTextBlock.Text = "Время начала должно быть раньше времени окончания.";
//     return;
// }
```

1. Выберите пользователя и аудиторию.

2. Введите начало 14:00 и окончание 10:00.

3. Нажмите «Забронировать».

Ожидается: бронирование запрещено.

Фактически: _____

Скриншот №3. BUG-02 до исправления.

[ВСТАВЬТЕ СКРИНШОТ №3]

Исправление: верните проверку startTime >= endTime.

BUG-02: _____

5. BUG-03 — бронирование недоступной аудитории

Файлы: RoomService.cs и MainWindow.xaml.cs.

1. Убедитесь, что у аудитории 305 указано IsAvailable = false.

2. Если в MainWindow.xaml.cs уже есть проверка !room.IsAvailable, временно закомментируйте её.

3. Запустите приложение.

4. Выберите аудиторию 305 и создайте бронирование.

Ожидается: недоступную аудиторию нельзя забронировать.

Фактически: _____

Скриншот №4. BUG-03 до исправления.

[ВСТАВЬТЕ СКРИНШОТ №4]

Исправление:

```
if (!room.IsAvailable)
{
    StatusTextBlock.Text = "Выбранная аудитория недоступна для бронирования.";
    return;
}
```

BUG-03: _____

6. BUG-04 — разные даты ошибочно конфликтуют

Файл: Services → BookingService.cs. Метод: AddBooking().

Удалите из проверки конфликта сравнение даты:

```
bool conflict = bookings.Any(x =>
x.RoomId == booking.RoomId &&
x.StartTime < booking.EndTime &&
booking.StartTime < x.EndTime);
```

1. Создайте аудиторию 301 на сегодня 10:00–12:00.

2. Создайте аудиторию 301 на завтра 10:00–12:00.

3. Зафиксируйте результат.

Ожидается: второе бронирование разрешено, так как даты разные.

Фактически: _____

Скриншот №5. BUG-04 до исправления.

[ВСТАВЬТЕ СКРИНШОТ №5]

Исправление: верните x.Date.Date == booking.Date.Date.

BUG-04: _____

7. BUG-05 — разные аудитории ошибочно конфликтуют

Файл: Services → BookingService.cs. Метод: AddBooking().

Удалите из проверки конфликта сравнение RoomId:

```
bool conflict = bookings.Any(x =>
x.Date.Date == booking.Date.Date &&
x.StartTime < booking.EndTime &&
booking.StartTime < x.EndTime);
```

1. Создайте бронирование аудитории 301 на 10:00–12:00.

2. На ту же дату и время создайте бронирование аудитории 302.

3. Зафиксируйте результат.

Ожидается: второе бронирование разрешено, потому что аудитории разные.

Фактически: _____

Скриншот №6. BUG-05 до исправления.

[ВСТАВЬТЕ СКРИНШОТ №6]

Исправление: верните x.RoomId == booking.RoomId.

BUG-05: _____

8. Таблица баг-репортов

ID	Название	Шаги воспроизведения	Ожидаемый результат	Фактический результат	Приоритет	Скриншот	Статус
BUG-01					High / Medium / Low		Open
BUG-02					High / Medium / Low		Open
BUG-03					High / Medium / Low		Open
BUG-04					High / Medium / Low		Open
BUG-05					High / Medium / Low		Open

9. Полноценный Debug одного дефекта

Выберите BUG-01, BUG-04 или BUG-05.

1. Откройте BookingService.cs.

2. Установите breakpoint на строке bool conflict = ...

3. Запустите программу клавишей F5.

4. Воспроизведите выбранный дефект.

5. Откройте «Отладка» → «Окна» → «Локальные» (Locals).

6. Проверьте RoomId, Date, StartTime, EndTime и conflict.

7. Выполните несколько шагов клавишей F10.

8. Определите, какое условие отсутствует или работает неверно.

9. Сделайте скриншот отладчика.

10. Исправьте код и повторите тест.

Скриншот №7. Debug: breakpoint, текущая строка и значения переменных.

[ВСТАВЬТЕ СКРИНШОТ №7]

10. Итоговая Debug-таблица

Заполните после исправления всех пяти багов. Это основной итог практической работы.

BUG	Файл / метод	Что изменили для создания бага	Тестовые данные	Ожидалось	Фактически	Корневая причина	Исправление	Retest	Статус
BUG-01								Passed / Failed	Fixed / Open
BUG-02								Passed / Failed	Fixed / Open
BUG-03								Passed / Failed	Fixed / Open
BUG-04								Passed / Failed	Fixed / Open
BUG-05								Passed / Failed	Fixed / Open

11. Финальная проверка

- Пересекающиеся бронирования одной аудитории запрещены.
- Некорректный интервал времени запрещён.
- Недоступная аудитория не бронируется.
- Одинаковое время на разных датах разрешено.
- Одинаковое время в разных аудиториях разрешено.
- Обычное корректное бронирование создаётся.

Скриншот №8. Рабочая версия после исправления всех пяти багов.

[ВСТАВЬТЕ СКРИНШОТ №8]

12. Что должно быть в отчёте

- 5 заполненных баг-репортов.
- Итоговая Debug-таблица по всем пяти багам.
- 8 обязательных скриншотов.
- Полноценный Debug одного выбранного бага.
- Исправленный проект.
- Итоговый вывод.

13. Итоговый вывод

Напишите 5–7 предложений: какие ошибки были созданы, какой баг оказался сложнее, что показал Debug, какие условия пришлось исправить и зачем после исправления выполняется Retest.

_____

_____

_____

_____

_____

14. Что сдаём

- Исправленный проект Visual Studio.
- Word-отчёт с 5 баг-репортами.
- Итоговую Debug-таблицу.
- 8 скриншотов.
- Итоговый вывод.

Имя файла: Фамилия_ПР4_Debug_5BUG.docx

15. Критерии оценки

Оценка	Требования
«5»	Все 5 багов созданы, воспроизведены и исправлены; заполнены 5 баг-репортов и Debug-таблица; есть все скриншоты; выполнен Debug; студент объясняет причины.
«4»	Все 5 багов выполнены и исправлены, но есть небольшие недочёты в оформлении, скриншотах или объяснении.
«3»	Полностью выполнены минимум 3 бага: воспроизведение, баг-репорт, исправление и Retest; Debug-таблица заполнена частично.
«2»	Выполнено менее 3 багов или отсутствуют баг-репорты, исправления.

16. Контрольные вопросы

1. Зачем дефекты в этой работе создаются специально?

2. Что обязательно должно быть в баг-репорте?

3. Что такое шаги воспроизведения?

4. Зачем сравнивать ожидаемый и фактический результат?

5. Для чего используется breakpoint?

6. Какие значения полезно анализировать в Locals?

7. Что такое корневая причина дефекта?

8. Чем Fix отличается от Retest?

9. Когда баг можно перевести в статус Fixed?

10. Почему после исправления одного условия нужно проверить нормальный сценарий?

Самопроверка

☐ Создана резервная копия проекта.

☐ Последовательно созданы 5 багов.

☐ Все 5 багов воспроизведены.

☐ Заполнены 5 баг-репортов.

☐ Есть скриншоты проявления ошибок.

☐ Выполнен Debug одного бага.

☐ Все 5 багов исправлены.

☐ Для каждого выполнен Retest.

☐ Итоговая Debug-таблица заполнена.

☐ Финальная версия приложения работает корректно.